

Guide enseignant / auteur, Atelier TP C

Ce guide couvre la surface d'administration et de configuration de l'Atelier TP C : le menu **Parametres**, le tuteur IA et sa commande, le mot de passe auteur, le mode enseignant « tout debloquer », l'outil de gestion des niveaux, les parcours, le diagnostic, Moodle et le stockage de la progression.

Il s'adresse à l'enseignant et à l'auteur de contenu. Pour la prise en main côté étudiant, voir `01-guide-utilisateur.md` ; pour le format détaillé des niveaux et de `meta.json`, voir `DOC-gestion-niveaux.md` ; pour le build et les sujets développeur, voir `04-doc-technique.md`.

Note ASCII : ce document est volontairement sans accents (il est aussi distribué en PDF dans le paquet). On écrit donc « enseignant », « parametres », « controle », etc.

1. Le menu Parametres

Le menu **Parametres** est en haut de la fenêtre. Il est construit par `fenetre._construire_menu`. Inventaire complet, dans l'ordre d'affichage :

Entree	Ce qu'elle fait	Handler
Changer de parcours...	Liste les parcours disponibles et memorise le choix pour le prochain lancement	<code>_changer_parcours</code>
Ouvrir le dossier du contenu	Ouvre l'Explorateur sur <code>contenu\<parcours courant></code>	<code>_ouvrir_dossier_contenu</code>
Emplacements et diagnostic...	Fenetre des chemins cles et de la presence des outils	<code>_ouvrir_diagnostic</code>
<i>(separateur)</i>		
Tuteur IA (case a cocher)	Active / desactive le tuteur IA pour la seance	<code>_basculer_tuteur</code>
Commande du tuteur...	Definit la ligne de commande qui invoque le moteur IA	<code>_changer_commande_ia</code>
<i>(separateur)</i>		
Gerer les niveaux...	Outil d'auteur : ajouter / modifier / retirer / reordonner les niveaux (mot de passe auteur)	<code>_ouvrir_gestion_niveaux</code>
Tout debloquer (mode enseignant) (case a cocher)	Ouvre toutes les etapes et tous les crans du tuteur (mot de passe auteur)	<code>_basculer_tout_debloque</code>
<i>(separateur)</i>		
Changer le mot de passe auteur...	Change le mot de passe qui protege le mode auteur	<code>_changer_mot_de_passe</code>

Deux entrees sont protegees par le mot de passe auteur (**Gerer les niveaux** et l'activation de **Tout debloquer**) ; les autres sont directes. En parcours **projet**, **Tout debloquer** est grise et **Gerer les niveaux** refuse de s'ouvrir (voir plus bas).

2. Le tuteur IA accepte n'importe quel outil en ligne de commande

C'est le point le plus important de ce guide, et le plus souvent meconnu : **le tuteur n'est pas limite a Claude ou a Codex**. Il sait invoquer n'importe quel executable en ligne de commande, sans modifier le code.

2.1 Comment le moteur est choisi

La selection est faite par `tuteur_ia._moteur_choisi()` , dans cet ordre de priorite :

1. **Commande sur mesure** (l'emporte sur tout le reste), lue par

`tuteur_ia.commande_personnalisee()` : - la variable d'environnement `ATELIER_AI_CMD` si elle

est definie (le temps d'une session, sans rien ecrire sur le disque) ; - sinon la commande enregistree dans `reglages.json`, cle `commande_ia` (`reglages.commande_ia()` / `definir_commande_ia()`), reglee via le menu **Parametres -> Commande du tuteur...** (`fenetre._changer_commande_ia`).

2. Sinon, la variable `ATELIER_AI` force un des moteurs connus (`claude` ou `codex`), a condition que son executable soit sur le PATH.
3. Sinon, **auto-detection** : le premier moteur connu trouve sur le PATH, dans l'ordre `claude` puis `codex` (`tuteur_ia._MOTEURS`).

Si aucun moteur n'est disponible, `_moteur_choisi()` renvoie `None` : l'atelier fonctionne alors sans tuteur (compiler, tester et lancer restent disponibles).

Garde-fou : meme pour une commande sur mesure, l'atelier verifie que le **premier mot** de la commande est un executable present sur le PATH (`shutil.which`). Une commande mal saisie se voit donc comme « pas de tuteur », pas comme une erreur au premier clic (le menu affiche l'avertissement « Commande introuvable »).

2.2 Comment la commande est construite

`tuteur_ia._commande()` fabrique la ligne effectivement lancee :

- Moteurs connus :
- `claude -> claude -p <prompt>`
- `codex -> codex exec --skip-git-repo-check <prompt>`
- Commande sur mesure : les morceaux sont decoupes facon shell (`shlex.split`), puis :
- si un morceau contient le marqueur `{prompt}`, la question y est substituee ;
- sinon la question est **ajoutee en dernier argument**.

Verifie sur ce depot (`_commande`) :

```
ATELIER_AI_CMD = "mon-moteur --sans-couleur {prompt}"
-> ['mon-moteur', '--sans-couleur', '<question>']

ATELIER_AI_CMD = "mon-moteur --flag"          (pas de marqueur)
-> ['mon-moteur', '--flag', '<question>']

claude -> ['claude', '-p', '<question>']
codex   -> ['codex', 'exec', '--skip-git-repo-check', '<question>']
```

Le prompt est construit par `construire_prompt` (energie de l'etape, style impose, cran d'aide, et, seulement si l'etudiant coche les cases, son code et la console). La reponse passe ensuite deux filtres anti-solution (`garde_fous.masquer_si_solution` puis `filtre_solution`) avant d'etre affichee. Ces filtres ne s'appliquent qu'aux moteurs connus comme au moteur sur mesure : brancher son propre outil ne contourne pas la bride.

2.3 Exemple concret : brancher un outil local quelconque

Supposons un script local `mon-tuteur` (deja sur le PATH) qui prend la question en dernier argument et repond sur la sortie standard.

- **Le temps d'une session** (variable d'environnement, rien de persistant) :

```
bat set ATELIER_AI_CMD=mon-tuteur --sans-couleur lancer.bat
```

- **De maniere durable** (ecrit dans `reglages.json`) : menu **Parametres -> Commande du tuteur...**, saisir par exemple :

```
mon-tuteur --sans-couleur {prompt}
```

Laisser le champ **vide** revient a l'auto-detection.

Contraintes a retenir :

- le **premier mot** doit etre un executable trouvable sur le PATH ;
- utilisez `{prompt}` si votre outil veut la question ailleurs qu'en dernier argument, sinon elle est simplement ajoutee a la fin ;
- l'outil doit repondre en **texte sur stdout**, en UTF-8, sans mode interactif (le tuteur ferme stdin et impose un delai de 120 s).

2.4 Activer / desactiver le tuteur

La case **Parametres -> Tuteur IA** ecrit `reglages.tuteur_actif` (default : actif). Desactive, l'atelier fonctionne entierement ; seules l'aide et la generation de code disparaissent. Utile pour une seance notee ou une salle sans reseau : plutot que de laisser des boutons qui repondent « moteur indisponible », on masque le tuteur.

A noter : reactiver le tuteur ne fait pas apparaitre un moteur. Si aucun n'est present, le menu le signale (« Tuteur active, moteur absent ») et invite a renseigner une commande.

3. Le mot de passe auteur

Gere par `auteur.py`. Il protege les operations d'auteur (voir sections 4 et 5) pour qu'un etudiant ne modifie pas le contenu par curiosite. **Ce n'est pas un secret de securite**, juste un garde-fou.

- **Default** : `auteur` (`auteur.MDP_DEFAULT`). A changer des que possible.
- Le mot de passe n'est **jamais stocke en clair** : seule son empreinte SHA-256 est gardees dans `auteur.json` (a la racine, **git-ignore**, local a chaque poste).
- Ordre de resolution de l'empreinte attendue (`auteur._hash_attendu`) : 1. `auteur.json` s'il existe ; 2. sinon la variable d'environnement `ATELIER_AUTEUR_MDP` (mot de passe en clair, hache a la volee), pratique pour un poste enseignant preconfigure ; 3. sinon le default `auteur`.

- **Changer le mot de passe** : menu **Parametres** -> **Changer le mot de passe auteur...** (`_changer_mot_de_passe`). Il faut d'abord saisir le mot de passe actuel, puis le nouveau ; l'empreinte est écrite dans `auteur.json` .

Ce que le mot de passe protege : **Gerer les niveaux** et l'**activation** de **Tout debloquer** (la desactivation, elle, n'en demande pas).

4. Tout debloquer (mode enseignant)

Case **Parametres** -> **Tout debloquer (mode enseignant)** (`fenetre._basculer_tout_debloque` , reglage `reglages.tout_debloque`).

Par default l'atelier est **progressif** : l'etudiant ouvre les exercices un a un, chacun se debloquant quand le precedent est valide, et les crans du tuteur se gagnent. Cocher cette case ouvre **toutes les etapes** et **les quatre crans du tuteur** d'un seul geste, comme en mode demo.

Points clés :

- **Activation protegee** par le mot de passe auteur ; la **desactivation** (retour au parcours progressif) n'en demande aucun.
- **Persistant** : le reglage est écrit dans `reglages.json` (cle `tout_debloque`) et survit au relancement. Pour revenir au parcours progressif, il suffit de **decocher**.
- **N'altère PAS la progression réelle**. Il ne fait que lever le verrouillage a l'affichage (`_remplir_liste`) et ouvrir les crans (`_maj_cran`). Aucune etape n'est faussement marquée « faite » : **rien de faux ne part vers Moodle**.
- En parcours **projet**, l'entree est **grisee** : tout y est deja ouvert (`setEnabled(self.mode != "projet")`).

Usages typiques : demonstration, seance de reprise, examen ou tout doit etre accessible.

5. Gerer les niveaux (outil d'auteur)

Menu **Parametres** -> **Gerer les niveaux...** (`_ouvrir_gestion_niveaux` , fenetre `dialogue_niveaux.DialogueNiveaux` , logique pure dans `gestion_niveaux.py`). Demande le mot de passe auteur. **Indisponible en parcours projet** (message d'information).

L'outil permet d'editer le contenu d'un parcours **depuis l'appli**, sans toucher aux fichiers a la main. Boutons de la fenetre :

- **Ajouter...** : formulaire (id, titre, mode, fichier edite, cran, noeud de cours, sortie attendue). Cree le dossier et ses gabarits, et insere le niveau en fin de parcours. L'appli propose de l'editer aussitot.
- **Modifier...** : editeur de contenu du niveau (onglets `enonce.md` , `starter.c` , `corrige.c` , plus le titre et la sortie attendue).

- **Retirer** : **detache** le niveau (le sort de `ordre` dans `parcours.json`). Le dossier n'est **pas efface** ; on peut le reattacher plus tard.
- **Monter / Descendre** : change la place du niveau dans le parcours.
- **Detaches...** : liste les niveaux detaches (dossier present, absent de `ordre`) et en reattache un.

Apres la fermeture, la liste du parcours dans l'appli est rechargee.

Rappel notation : si l'edition touche le **parcours note** (`be_c` par default), l'outil realigne `compagnon/etapes_notees.json` sur le nouvel ordre et rappelle que **le compagnon doit etre redeploye** pour que les notes en tiennent compte. Voir `DOC-gestion-niveaux.md`.

Pour le format complet d'un niveau (fichiers d'un dossier, tous les champs de `meta.json`, gabarits, sortie attendue) : voir `DOC-gestion-niveaux.md`. Ce guide ne duplique pas le tableau detaille.

6. Parcours : changer, ajouter, et le piege du paquet livre

Un **parcours** est une suite d'exercices. C'est un **sous-dossier de** `contenu\` qui contient un fichier `parcours.json`. La decouverte est automatique : `diagnostic.parcours_disponibles()` liste tout sous-dossier de `contenu\` porteur d'un `parcours.json` (un dossier sans ce fichier est ignore).

`parcours.json` contient :

- `ordre` : la liste des noms de dossiers d'exercices, dans l'ordre d'affichage ;
- `mode` : `"isole"` (exercices independants, le cas courant) ou `"projet"` ; default `"isole"` ;
- `libre` : optionnel, default `false`. A `true`, toutes les etapes sont ouvertes d'emblee, sans franchir les portes precedentes (utile pour un parcours de revision).

Changer de parcours : menu **Parametres -> Changer de parcours...**. Le choix est memorise dans `reglages.json` (`reglages.definir_parcours`). Il prend effet **au prochain lancement** (fermer puis relancer).

Ajouter un parcours : deposer un dossier dans `contenu\` avec un `parcours.json` et les dossiers d'exercices cites dans `ordre`. Le plus simple est de copier un exercice existant de `be_c` et de l'adapter (format detaille dans `DOC-gestion-niveaux.md`).

6.1 NUANCE CRITIQUE : le paquet livre force `be_c`

Le parcours reellement ouvert est resolu par `atelier_snake._parcours_choisi()` : **priorite a l'argument `--parcours`, sinon `reglages.dernier_parcours()`** (default `be_c`). D'ou une difference importante selon le lanceur :

- **Depuis les sources** (`Atelier.bat` , qui lance `atelier_snake.py` **sans** `--parcours`) : le choix du menu **est pris en compte** au prochain lancement. Cas nominal.
- **Depuis l'exé livre** : le point d'entrée `packaging/entree_be_c.py` **force** `--parcours be_c` , et le lanceur `packaging/lancer.bat` passe déjà `--parcours be_c` . Comme `--parcours` est prioritaire sur `reglages.json` , **le choix du menu n'a aucun effet dans le paquet livre** : l'exé ouvre toujours `be_c` .

Pour ouvrir un autre parcours dans le livrable, il faut **éditer** `lancer.bat` et remplacer `--parcours be_c` par `--parcours <nom>` , puis relancer.

6.2 Ou vit le contenu, et ce que le build embarque

- **Sources / checkout** : `<repo>\contenu\<nom>\` .
- **Exe PyInstaller livre** : `TP-C-perso\_internal\contenu\<nom>\` (les modules Python vivent dans `_internal\`). Un parcours ajoute au produit livre se dépose là.

Le build **n'embarque que** `be_c` (`packaging/build_windows.ps1` , ligne `--add-data "$Repo\contenu\be_c;contenu/be_c"`). Pour livrer un autre parcours, soit le copier après coup dans `_internal\contenu\` , soit ajouter une ligne `--add-data` au `packaging`. Voir `04-doc-technique.md` pour le build.

Ce comportement est déjà documenté dans `GUIDE.md` et `DOC-gestion-niveaux.md` ; ce guide est cohérent avec eux.

7. Emplacements et diagnostic

Menu **Paramètres -> Emplacements et diagnostic...** (`dialogue_diagnostic.py` , logique dans `diagnostic.py`). C'est la version fenêtre de l'ancien `diagnostic.bat` .

La fenêtre montre :

- **Emplacements clés** (`diagnostic.chemins_cles`), chacun avec un bouton « Ouvrir le dossier » : Dossier de l'appli, Contenu du parcours, Progression, Compilateur w64devkit.
- **Outils** (`diagnostic.outils`), état **présent / absent** en couleur :
- `gcc` (compile et ouvre les portes ; **nécessaire**),
- `clangd` (diagnostics live ; optionnel),
- `claude` (tuteur IA ; optionnel).

La détection cherche l'outil sur le PATH puis dans `w64devkit\bin` embarqué. Note : seul `claude` est sondé ici ; un tuteur branche sur `codex` ou sur une commande sur mesure ne se reflète pas dans cette liste, sans que cela empêche le tuteur de marcher.

- Un bouton « **Créer un raccourci sur le bureau** » (`_creer_raccourci`) qui pose un fichier `Atelier TP C.lnk` pointant sur le lanceur (`Atelier.bat`), via `WScript.Shell`.
-

8. Moodle (optionnel)

Pont facultatif vers un compagnon Moodle (`moodle_sync.py` , bouton dans `fenetre.py`). **L'atelier fonctionne pleinement hors ligne** ; Moodle n'est active que si on le demande.

- Sous le panneau PARCOURS, un bouton « **Connecter a Moodle** » (`_connecter_moodle`). On y colle le **code** affiche par l'activite Moodle du TP ; l'appli l'echange contre un jeton permanent (`moodle_sync.appairer`) range dans `moodle_sync.json` .
- Une fois appaire, chaque porte franchie est mise en file locale puis **envoyee automatiquement** au compagnon ; un envoi rate attend le prochain rejeu. La progression deja acquise avant la connexion est renvoyee (`signaler_deja_faits`), et la reprise multi-poste fusionne les etapes deja validees cote compagnon.
- Interrupteur franc : la remontee n'est active que si `ATELIER_SUIVI` vaut `moodle` . Le **default est local** (aucun reseau) ; le bundle distribue ne parle donc a aucun serveur tant que personne ne l'a demande.

Limitation connue (voir `SUIVI-MOODLE-TEST-LINUX-2026-07-09.md`) : un score ne se pose dans le **carnet de notes** Moodle que pour un **vrai etudiant inscrit** au cours. Un compte enseignant n'a pas de ligne notable dans le carnet ; l'echange fonctionne mais la case reste vide. La confirmation visuelle du chiffre se fera au premier vrai etudiant.

9. Stockage de la progression et des reglages

- `progression.json` : la progression reelle de l'etudiant (etapes validees, cran disponible). Dans l'exemple, il est ecrit dans `_internal\` : **re-extraire le zip par-dessus efface donc la progression**. A la racine du depot en mode source.
- `reglages.json` : dernier parcours, tuteur actif, commande du tuteur, tout debloque. **Local et git-ignore.**
- `auteur.json` : empreinte du mot de passe auteur. **Local et git-ignore.**
- `moodle_sync.json` : appairage et file d'attente Moodle. Local.

Releve de progression : un releve lisible existe (`releve.py`), disponible **en ligne de commande** via `atelier_snake.py --releve` (ecrit `releve.txt` et l'affiche). Il liste les etapes du parcours, le nombre de validees sur le total, le pourcentage, et une empreinte de controle. C'est un outil hors ligne (mode local), pas un bouton de la fenetre.

Recapitulatif : ou est quoi

Sujet	Fichier(s)
Menu Parametres et branchements	<code>fenetre.py</code> (<code>_construire_menu</code> + handlers)
Tuteur IA, choix du moteur, commande	<code>tuteur_ia.py</code> , <code>reglages.py</code>
Mot de passe auteur	<code>auteur.py</code> , <code>auteur.json</code> (local)
Tout debloquer / tuteur actif / parcours	<code>reglages.py</code> , <code>reglages.json</code> (local)
Gestion des niveaux	<code>dialogue_niveaux.py</code> , <code>gestion_niveaux.py</code> , <code>DOC-gestion-niveaux.md</code>
Parcours (decouverte, chemins)	<code>diagnostic.py</code> , <code>chemins.py</code> , <code>atelier_snake.py</code>
Diagnostic (fenetre)	<code>dialogue_diagnostic.py</code> , <code>diagnostic.py</code>
Moodle	<code>moodle_sync.py</code> , <code>fenetre.py</code> (<code>_connecter_moodle</code>)
Progression / releve	<code>progression.py</code> , <code>releve.py</code>
Paquet livre (forcage <code>be_c</code>)	<code>packaging/entree_be_c.py</code> , <code>packaging/lancer.bat</code>

Pour aller plus loin : `DOC-gestion-niveaux.md` (format des niveaux), `GUIDE.md` (installation et vue d'ensemble), `01-guide-utilisateur.md` (etudiant), `04-doc-technique.md` (build et developpement).